

第五章 近端策略优化

重要性采样

先回顾一下上一章我们是怎么采样然后优化策略的：先用当前策略 π_θ 运行多次，采样得到多组数据，然后根据这些数据将 π_θ 优化为 $\pi_{\theta_{\text{new}}}$ 。然后以 $\pi_{\theta_{\text{new}}}$ 作为当前策略继续执行上面的循环。

这样做有什么问题？**效率低下**：原策略 π_θ 采了很多数据，**只用来更新了一次就废弃了**，而采样本身的开销要远大于根据采集的数据更新策略，所以造成了很大的浪费。

我们希望：即使策略已经被更新，我们仍然能够**复用之前采集的数据来进行策略更新**。

这里也就是**同策略**和**异策略**的区别：上一章的方法中，采集数据的策略和即将更新的策略是同一个，因此称为同策略；而我们接下来要做的是将两个策略分开：旧策略负责采样，更新的是新策略，因此称为异策略。

但是遇到问题：**策略改变之后，旧策略的数据不能直接套用到新策略的身上**，而是需要经过一次匹配修正：这就是**重要性采样**。

结合上一章里把 R 换成优势函数 A 的做法，我们可以得到：

$$J^{\pi_{\text{old}}}(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\text{old}}} \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} A^{\pi_{\text{old}}}(s_t, a_t) \right]$$

这里 A 代表在旧策略下得到的优势函数，而前面的 $\frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ 则是将旧策略下的数据转换为新策略下。整个式子的意思就是：**利用旧策略的数据，计算得到了当前新策略下的奖励**。

对 θ 求梯度，得到：

$$\nabla_\theta J^{\pi_{\text{old}}}(\theta) = \mathbb{E}_{\pi_{\text{old}}} \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} A_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right]$$

记 $\frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ 为 $\rho_t(\theta)$ 。多次采样平均代替期望，有

$$\nabla_\theta \hat{J}(\theta) = \frac{1}{N} \sum_{t=1}^N \rho_t(\theta) A_t \nabla_\theta \log \pi_\theta(a_t | s_t)$$

对比上一章我们得到的

$$\nabla_\theta J(\theta) \approx \sum_t A(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t)$$

两者只相差一个 $\rho_t(\theta)$ （也就是两种策略的转移），其他的操作流程和计算都是一样的。

$1/N$ 的有无其实没有太大影响，因为只是相差常数倍，没有改变改进方向。

总结一下这一节：旧策略先与环境交互，得到状态、动作和奖励；根据这批数据计算优势；训练时把同一个状态动作对同时输入旧策略和当前策略，计算新旧动作概率比；再用“概率比 $\times$ 优势”构造目标函数，对当前策略求梯度并更新。当前策略可以在这批旧数据上更新多次，更新结束后成为新的旧策略，再重新与环境采样。

近端策略优化 PPO

上一节里面我们实现了旧策略采集数据的复用。那我们能不能只用旧策略采集的这一点数据不断更新新策略，达到一个比较好的结果呢？显然**不行**（想得太美了）。

有什么问题呢？多次用旧数据更新策略会导致**严重的参数极端化**：某些 ρ 接近 0 而另一些极大，使得**少数样本主导梯度**，旧数据对新策略的代表性越来越差，训练变得**不稳定**。

真正可取的做法：旧策略采集一批数据，根据这些数据更新几轮新策略；当新策略与旧策略**差距过大时**，把当前的新策略当成旧策略去采样，再用现在的样本更新新一轮的新策略。

简单来讲就是两层循环：**内层循环负责更新新策略；外层循环负责更新老策略**。

此外，还有一个问题：更新幅度。我们需要找到一个方案，鼓励好动作、抑制坏动作，但**不允许新策略相对于旧策略变化得过分激进**。

首先我们需要一个标尺来**衡量两个策略之间的差异**：**KL 散度**，其公式如下所示：

$$\text{KL}(\pi_{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) = \sum_a \pi_{\text{old}}(a | s) \log \frac{\pi_{\text{old}}(a | s)}{\pi_{\theta}(a | s)}$$

这是关于某一状态的 KL 散度。两策略的 KL 散度即各状态取平均得到。**散度越大，策略之间的差异越大**。

KL 散度的取值范围为 $[0, +\infty)$ ，当且仅当两策略完全相同时取 0。

举个例子：对于状态 s ，有三种动作 $[a_1, a_2, a_3]$ ，现有三个策略 π_1, π_2, π_3 ，分别有

$$\begin{aligned}\pi_1(\cdot | s) &= [0.7, 0.2, 0.1], \\ \pi_2(\cdot | s) &= [0.6, 0.3, 0.1], \\ \pi_3(\cdot | s) &= [0.1, 0.1, 0.8].\end{aligned}$$

则显然对于该状态 s ，有 $\text{KL}(\pi_1 \parallel \pi_2) < \text{KL}(\pi_1 \parallel \pi_3)$ 。

限制策略过度更新，提出了三个循序渐进的方案：

1. TRPO 信赖区域策略优化（PPO 的前身）。

核心思想：要求 $\text{KL}(\pi_{\text{old}}, \pi_{\theta}) < \delta$ ，其中 δ 人为给定，是一个规定的范围。

问题：这是一个**硬约束**，工程实践中不方便处理。

2. PPO 近端策略优化，分为两个变体。

1. PPO-penalty。

核心思想：把 TRPO 里面的**硬约束换成软约束**，调整目标函数：

$$J_{\text{PPO}}(\theta) = \mathbb{E}[r_t(\theta)A_t] - \beta \text{KL}(\pi_{\text{old}}, \pi_{\theta})$$

其中 β 为 KL 惩罚强度。

缺点：要计算 KL，还需要调 β 的值。

2. PPO-clip。

核心思想：不在目标函数中显式引入 KL，而是直接**限制重要性采样比值 ρ** ：

$$J^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(\rho_{\theta} A_t, \text{clip}(\rho_{\theta}, 1 - \epsilon, 1 + \epsilon) A_t)]$$

其中 $\text{clip}(\rho_{\theta}, 1 - \epsilon, 1 + \epsilon)$ 表示将 ρ_{θ} 裁剪到 $(1 - \epsilon, 1 + \epsilon)$ 区间内。

通过**直接裁剪过于大的 ρ** 来限制每一次更新的变化幅度。

本节采用 PPO 的方法限制了过于剧烈的策略变动，增强了学习的稳定性，对于执行流程没有任何影响。